

B4 課題 仮想環境セットアップガイド

uv を使った Python 環境構築と
JupyterLab の起動

前提：マニュアル第 1 弾（WSL 入門）完了済み

2026 ver.

目次

第 1 章	はじめに	4
1.1	このガイドでやること	4
1.2	ゴール	4
1.3	前提条件	4
第 2 章	仮想環境とは何か	6
2.1	なぜ仮想環境が必要か	6
2.1.1	バージョン衝突の問題	6
2.1.2	グローバル環境を汚さないために	6
2.2	仮想環境の仕組み	7
2.3	仮想環境の実体	7
2.4	仮想環境を使うメリット	8
第 3 章	uv の仕組みを理解する	9
3.1	uv の概要	9
3.2	pip + venv との比較	9
3.3	pyproject.toml とは	10
3.3.1	ファイルの構造	10
3.3.2	バージョン指定の書き方	11
3.3.3	パッケージを追加するには	11
3.4	uv.lock とは	12
3.4.1	なぜ必要か	12
3.4.2	uv.lock が解決すること	12
3.4.3	uv.lock の管理ルール	13
3.5	uv sync の動作	13
第 4 章	uv のインストール	15
4.1	インストールコマンド	15

4.2	PATH の反映	15
4.3	インストールの確認	16
4.4	uv のアップデート (将来の参考用)	16
第 5 章	フォルダ構成と配置場所	17
5.1	配置先のディレクトリ	17
5.2	重要なファイルの説明	18
第 6 章	課題リポジトリを取得する	19
6.1	git clone で取得する	19
6.2	2 回目以降: 最新版に更新する	20
第 7 章	仮想環境のセットアップ	21
7.1	フォルダに移動する	21
7.2	uv sync を実行する	21
7.2.1	実行中の出力例	22
7.2.2	2 回目以降の実行	22
7.3	セットアップの確認	22
第 8 章	ノートブックの実行方法	24
8.1	VS Code でノートブックを開く (推奨)	24
8.1.1	事前準備: Jupyter 拡張機能のインストール	24
8.1.2	フォルダを開いてノートブックを実行する	24
8.1.3	セルの実行	25
8.2	課題ノートブックの一覧	25
8.3	JupyterLab を使う場合 (任意)	26
第 9 章	よくあるエラーと対処	27
9.1	uv: command not found	27
9.2	No pyproject.toml found	27
9.3	JupyterLab でカーネルが見つからない	28
9.4	ModuleNotFoundError が出る	28
9.5	uv sync がネットワークエラーで止まる	28
9.6	VS Code で「カーネルを起動できませんでした」と表示される	29
9.7	Jupyter で「Python 3.12.x が使用できなくなった」と表示される	30
9.8	uv sync 後に Permission denied が出る	30
9.9	ブラウザが自動で開かない	31
第 10 章	補足知識	32

10.1	.venv/ フォルダの扱い	32
10.2	環境を作り直す	32
10.3	新しいパッケージを追加する	32
10.4	スクリプトを直接実行する	33
10.5	MATLAB とのデータ互換	33
10.6	インストール済みパッケージの確認	34
第 11 章	セットアップ手順まとめ (チートシート)	35
11.1	最初の 1 回だけ行う作業	35
11.2	毎回の作業	35
11.3	よく使うコマンドまとめ	36
11.4	トラブル時の確認フロー	36
第 12 章	研究での活用：uv init / activate / VS Code	37
12.1	uv init：新しいプロジェクトを作る	37
12.1.1	パッケージを追加する	37
12.1.2	パッケージを削除する	38
12.2	Python スクリプトを実行する 3 つの方法	38
12.2.1	方法 (1) uv run (推奨)	38
12.2.2	方法 (2) activate してから実行	39
12.2.3	方法 (3) .venv/bin/python を直接指定	39
12.3	VS Code との連携	40
12.3.1	インタープリタの選択	40
12.3.2	VS Code の統合ターミナルで実行する	40
12.3.3	Python ファイルを「実行」ボタンで動かす	40
12.3.4	Jupyter Notebook を VS Code で開く	41
12.4	研究プロジェクトの典型的な流れ	41
12.5	よく使う uv コマンド一覧	42
12.5.1	Python バージョン管理	42
12.5.2	依存関係の確認	42
12.5.3	requirements.txt へのエクスポート	43
12.5.4	ロックファイルだけを更新する	43
12.5.5	uvx：ツールを一時的に実行する	43
12.5.6	キャッシュの管理	43
付録 A	参考リンク集	45
A.1	uv 公式ドキュメント	45
A.2	本マニュアルとの対応	45

第 1 はじめに

このガイドは、**マニュアル第 1 弾（WSL 入門）の続編**です。WSL（Ubuntu）のインストールとターミナルの基本操作は完了している前提で進めます。

1.1 このガイドでやること

- 仮想環境とは何か、なぜ必要かを理解する
- パッケージマネージャ **uv** をインストールする
- `git clone` で課題リポジトリを取得する
- `uv sync` で必要なライブラリをまとめてインストールする
- VS Code でノートブックを開いて課題を実行する

1.2 ゴール

このガイドを読み終えると、VS Code で課題フォルダを開くだけで作業を始められるようになります。

```
$ code ~/NMR-lab-kadai年度/2026課題引継ぎ_B4
```

1.3 前提条件

- WSL（Ubuntu）がインストール済みであること
- WSL のターミナルを開けること
- インターネットに接続できること（初回セットアップ時のみ）
- VS Code に Jupyter 拡張機能がインストールされていること

メモ

Python のインストールは不要です。

uv が Python 本体も含めて自動でダウンロード・管理します。

第2 仮想環境とは何か

2.1 なぜ仮想環境が必要か

Python でプログラムを書くとき、numpy や matplotlib などの外部ライブラリ（パッケージ）を使います。これらには**バージョン**があり、プロジェクトによって必要なバージョンが異なります。

2.1.1 バージョン衝突の問題

次のようなシナリオを考えてみましょう。

B4 課題（NMR ラボ）	別の研究プログラム
numpy 1.26 が必要	2.0 が必要

同じ Python 環境に両方をインストールしようとする、片方が上書きされてどちらかが動かなくなります。

状況	結果
numpy 1.26 をインストール	別の研究プログラムが動かない
numpy 2.0 にアップグレード	B4 課題が動かなくなる

これが**依存関係の衝突**です。

2.1.2 グローバル環境を汚さないために

pip install を繰り返すと、システム全体の Python 環境にパッケージがどんどん蓄積されます。

- 使わなくなったパッケージが残り続ける
- バージョンアップで他のプログラムが突然壊れる

- 「先輩の PC では動いたのに自分の PC では動かない」が起きやすい

2.2 仮想環境の仕組み

仮想環境 (Virtual Environment) はプロジェクトごとに独立した Python の箱を作る仕組みです。

仮想環境のイメージ

システム全体

- └ Ubuntu のシステム Python (素の状態)
- └ B4 課題用 仮想環境 (.venv)
 - | numpy 1.26, bm3d 4.0, jupyterlab 4.0 ...
- └ 別プロジェクト用 仮想環境 (.venv)
 - | numpy 2.0, tensorflow ...

各仮想環境は**完全に独立**しており、一方を変更しても他方には影響しません。

2.3 仮想環境の実体

仮想環境はただのフォルダです。プロジェクトフォルダの中に `.venv/` というフォルダが作られ、その中にライブラリの実体が格納されます。

年度

```
2026 課題引継ぎ_B4/ |——
  pyproject.toml      # どのライブラリが必要か書いてある |——
  uv.lock             # バージョンを固定するファイル |——
  .venv/              # 仮想環境 (uv sync で自動生成) | |——
    bin/ | |——
      lib/python3.11/site-packages/ | |——
        numpy/ | |——
        matplotlib/ | |——
        ... |——
  No.1_FID/ |——
  ...
```

ポイント

`.venv/` の中身は自動生成されるため、自分で触る必要はありません。 `uv sync` を実行するだけで作られます。

2.4 仮想環境を使うメリット

依存関係の分離 プロジェクトごとに必要なパッケージだけをインストールできる

再現性 全員が同じバージョンのライブラリを使えるので「自分の環境では動くのに.....」
という問題が起きない

削除が簡単 不要になったら `.venv/` フォルダを削除するだけ

システムを汚さない Ubuntu のシステム Python は変更しないので OS が安定する

第3 uv の仕組みを理解する

3.1 uv の概要

uv（読み方：ユーブイ）は Astral 社が開発した Python パッケージマネージャです。公式サイト：<https://docs.astral.sh/uv/>

従来の **pip**（パッケージインストーラー）と **venv**（仮想環境作成ツール）の両方の機能を一つに統合しており、以下の特徴があります。

高速 Rust で書かれており、**pip** より 10～100 倍速い。多数のパッケージをインストールしても数秒で完了する

再現性が高い `uv.lock` ファイルで全員がまったく同じバージョンを使える

Python も自動管理 Python 本体のインストールも自動で行う。事前に Python をインストールしておく必要がない

シンプル `uv sync` 一発で仮想環境の作成からパッケージのインストールまで完了する

3.2 pip + venv との比較

従来の方法（`pip + venv`）と `uv` を比べると次のとおりです。

操作	pip + venv	uv
Python のインストール	別途必要	自動
仮想環境を作る	<code>python3 -m venv .venv</code>	自動 (uv sync で)
仮想環境を有効化	<code>source .venv/bin/activate</code>	不要
パッケージをインストール	<code>pip install -r req.txt</code>	<code>uv sync</code>
スクリプトを実行	<code>python script.py</code>	<code>uv run python script.py</code>
パッケージを追加	<code>pip install xxx</code>	<code>uv add xxx</code>
新規プロジェクト作成	手動で <code>pyproject.toml</code> 作成	<code>uv init</code>
速度	普通	10~100 倍速い

activate が不要な理由

uv では `source .venv/bin/activate` による仮想環境の有効化が不要です。uv `run` をつけるだけで、自動的に `.venv/` 内の Python とライブラリを使って実行してくれます。「有効化し忘れてシステム Python で実行してしまった」というよくあるミスがなくなります。

3.3 pyproject.toml とは

`pyproject.toml` はプロジェクトの設定ファイルです。「このプロジェクトには何が必要か」をまとめて書く場所です。

3.3.1 ファイルの構造

B4 課題フォルダの `pyproject.toml` を見てみましょう。

Listing 3.1 `pyproject.toml` の構造

```
[project]
name = "nmrlab-b4-2026"      # プロジェクトの名前
version = "2026.1"          # バージョン
requires-python = ">=3.11"   # Python 3.11 以上が必要
```

```
dependencies = [                                # 必要なライブラリの一覧
    "numpy>=1.26",                             # numpy バージョン 1.26 以上
    "scipy>=1.12",
    "matplotlib>=3.8",
    "scikit-image>=0.22",
    "PyWavelets>=1.5",
    "bm3d>=4.0",
    "openpyxl>=3.1",
    "jupyterlab>=4.0",
    "ipywidgets>=8.0",
    "japanese-matplotlib>=1.1",
]

[tool.uv]
dev-dependencies = [                          # 開発時だけ使うツール（通常不要）
    "pytest>=8.0",
]
```

3.3.2 バージョン指定の書き方

書き方	意味
"numpy>=1.26"	1.26 以上ならどのバージョンでも OK
"numpy==1.26.4"	1.26.4 ぴったり（固定）
"numpy>=1.26,<2.0"	1.26 以上かつ 2.0 未満
"numpy"	バージョン指定なし（最新）

pyproject.toml は「希望」を書く場所

pyproject.toml の dependencies は「このバージョン以上なら動く」という**条件**を書くところです。実際にインストールする**正確なバージョン**は uv.lock に記録されます。

3.3.3 パッケージを追加するには

pyproject.toml を直接編集してもよいですが、uv add コマンドを使うと自動で追記されます。

```
$ uv add pandas
```

実行すると `pyproject.toml` の `dependencies` に `"pandas>=X.X.X"` が自動追記されます。

3.4 uv.lock とは

`uv.lock` は `uv` が自動生成する**ロックファイル**です。**絶対に手動で編集してはいけません**。

3.4.1 なぜ必要か

`pyproject.toml` には「numpy 1.26 以上」と書きましたが、これだけでは「今インストールしたら 1.26.4 なのか 1.27.0 なのか」がわかりません。

時間が経つと最新バージョンが変わるため、同じ `pyproject.toml` を使ってもインストールのタイミングによって入るバージョンが変わります。

ロックファイルがないと起きる問題

自分（3 月にインストール）：numpy 1.26.4
後輩（9 月にインストール）：numpy 1.27.2
→ 同じコードが異なる環境で動く可能性がある

3.4.2 uv.lock が解決すること

`uv.lock` には**全パッケージの正確なバージョンとハッシュ値**が記録されます。

```
version = 1
requires-python = ">=3.11"

[[package]]
name = "numpy"
version = "1.26.4"      # 正確なバージョンが固定されている
source = { registry = "..."}
sdist = { url = "...", hash = "sha256:abc123..." }
```

`uv sync` は `uv.lock` を読んで**全員が同じバージョン**をインストールします。誰がいつ実行しても同じ環境になります。

3.4.3 uv.lock の管理ルール

- 手動で編集しない (uv が自動管理する)
- 共有フォルダやリポジトリに含める (全員で共有する)
- uv add/uv remove を実行すると自動更新される
- uv sync を実行しても uv.lock は変化しない (uv.lock に従ってインストールするだけ)

pyproject.toml と uv.lock の関係まとめ

	pyproject.toml	uv.lock
書いてあること	「これ以上のバージョンが必要」という条件	実際にインストールする正確なバージョン
誰が書く	人間 (uv add でも可)	uv が自動生成
手動編集	OK	してはいけない
共有する	する	する

3.5 uv sync の動作

uv sync は以下のことを**すべて自動**で行います。

1. pyproject.toml を読んで必要な Python バージョンを確認
2. 必要な Python がなければ自動ダウンロード・インストール
3. .venv/ フォルダを作成 (すでにあれば再利用)
4. uv.lock を読んで記録されたバージョンのパッケージを .venv/ にインストール
5. 余分なパッケージ (uv.lock にないもの) があれば削除

uv sync の出力例 (初回)

```
$ uv sync
Using CPython 3.11.x          <- Python を自動選択
Creating virtual environment at: .venv  <- .venv を作成
Resolved 47 packages in 0.5ms
Installed 47 packages in 8.2s  <- ここが pip より圧倒的に速い
+ bm3d==4.0.2
+ numpy==1.26.4
+ jupyterlab==4.3.6
```

```
...
```

```
$ uv sync
Resolved 47 packages in 0.5ms
Audited 47 packages in 1ms    <- 確認のみ、インストールなし
```

uv sync は「冪等」

何度実行しても結果は同じです。すでに正しい状態なら何もしません。環境がおかしいと感じたらとりあえず uv sync を試してください。

第4 uv のインストール

4.1 インストールコマンド

WSL のターミナルを開いて以下を実行します。これは最初の 1 回だけでよいです。

```
$ curl -LsSf https://astral.sh/uv/install.sh | sh
```

実行すると以下のような出力が出ます。

```
downloading uv 0.6.x x86_64-unknown-linux-musl
installing to /home/username/.local/bin
uv
uvx
everything's installed!

To add $HOME/.local/bin to your PATH, either
restart your shell or run:

source $HOME/.bashrc
```

`curl ... | sh` について

インターネットからスクリプトをダウンロードして直接実行するコマンドです。
Astral 社の公式インストーラーなので安全です。

4.2 PATH の反映

インストール直後はコマンドがまだ認識されません。ターミナルを一度閉じて開き直してください。

あるいは、ターミナルを開き直さずに即座に反映させる場合は：

```
$ source ~/.bashrc
```

4.3 インストールの確認

以下のコマンドでバージョンを確認します。

```
$ uv --version
uv 0.6.x (linux x86_64 ...)
```

このように表示されれば OK です。

```
$ uv --version
-bash: uv: command not found
```

command not found と表示された場合は、ターミナルの再起動か `source ~/.bashrc` を試してください。それでも解決しない場合は第 9 章を参照してください。

4.4 uv のアップデート（将来の参考用）

uv 自体のアップデートは以下のコマンドで行えます。セットアップ時に実行する必要はありません。

```
$ uv self update
```

第5 フォルダ構成と配置場所

5.1 配置先のディレクトリ

`git clone` を実行すると、ホームディレクトリ直下にリポジトリフォルダが作成されます。課題フォルダはリポジトリ直下の 2026 年度_B4 課題引継ぎ/ にあります。

フォルダ構成

```

/home/username/          <- ホームディレクトリ (~ と同じ) |——
NMR-lab-kadai/           <- git clone で作成される |——年度
  2026課題引継ぎ_B4/      <- VS Code で開くフォルダ | |——
    .vscode/ | | |——
      settings.json      <- カーネル自動設定 | |——
    pyproject.toml | |——
    uv.lock | |——
  No.1_FID/ | | |——
    makesignal.ipynb | | |——
    heikatsu.ipynb | |——
  No.2_FFT1D/ | | |——
    FFT1D.ipynb | | |——
    kukei.ipynb | | |——
    sft.ipynb | |——
  No.3_FFT2D/ | | |——
    FFT2D.ipynb | |——
  No.4_DFT_exercise/ | | |——
    DFT_template.ipynb | |——
  No.5_CompressedSensing/ | | |——
    cs_tv.ipynb | | |——
    cs_wavelet.ipynb | |——
  No.6_Denoising/ | | |——
    add_noise.ipynb | | |——
    denoise_bm3d.ipynb | |——
  No.7_Metrics/ | | |——

```

```

psnr_ssim.ipynb | ───
No.8_CT/ | ───
ct_pipeline.ipynb ───
uv-manual/          <- セットアップマニュアル

```

username について

username の部分は自分の WSL ユーザー名です。次のコマンドで確認できます。

```
$ whoami
```

5.2 重要なファイルの説明

ファイル	役割
pyproject.toml	必要なライブラリとバージョン条件が書かれた設定ファイル。uv sync はこれを読んでインストール内容を決める
uv.lock	実際にインストールするバージョンを厳密に記録したファイル。全員が同じ環境を再現するために使う。 編集しないこと
.venv/	uv sync を実行すると自動生成される仮想環境フォルダ。git には含まれないので各自が uv sync で作成する
.vscode/settings.json	VS Code が自動で .venv の Python を選ぶための設定ファイル。カーネル選択の手間を省く
*.ipynb	JupyterLab で開く課題ノートブック
*.py	対応する Python スクリプト（ノートブックと同じ処理）

.venv/ は git 管理外

.venv/ フォルダはサイズが数百 MB あり、uv sync で誰でも再生成できるため git には含まれていません。git clone 後に各自で uv sync を実行してください。

第 6 課題リポジトリを取得する

課題ファイルは GitHub 上で管理されています。git clone コマンドで WSL のホームディレクトリに取得します。**この操作は最初の 1 回だけ行います。**

6.1 git clone で取得する

WSL ターミナルで以下を実行します。

```
$ cd ~  
$ git clone \  
https://github.com/shiru2/NMR-lab-kadai.git
```

正常に完了すると NMR-lab-kadai フォルダが作成されます。

```
$ ls \  
~/NMR-lab-kadai年度/2026課題引継ぎ_B4/  
No.1_FID No.2_FFT1D No.3_FFT2D  
No.4_DFT_exercise No.5_CompressedSensing  
No.6_Denoising No.7_Metrics No.8_CT  
README.md pyproject.toml uv.lock
```

pyproject.toml と uv.lock が表示されれば正しく取得できています。

git が使えない場合

大学のネットワーク環境によって git clone がブロックされる場合があります。その場合は GitHub のページから zip をダウンロードして展開してください。

Code → Download ZIP を選び、展開したフォルダを

\\wsl.localhost\Ubuntu\home\username

に配置してください (username は自分の WSL ユーザー名)。

6.2 2 回目以降：最新版に更新する

課題ファイルが更新された場合は `git pull` で最新版を取得できます。

```
$ cd ~/NMR-lab-kadai  
$ git pull
```

第 7 仮想環境のセットアップ

7.1 フォルダに移動する

まず課題フォルダに移動します。

```
$ cd \  
~/NMR-lab-kadai年度/2026課題引継ぎ_B4
```

移動できたか確認します。

```
$ pwd  
/home/username/NMR-lab-kadai年度/2026課題引継ぎ_B4
```

このように表示されれば OK です。

7.2 uv sync を実行する

```
$ uv sync
```

初回実行時は Python のダウンロードと全ライブラリのインストールが自動で行われます。
回線速度によっては数分かかる場合があります。

7.2.1 実行中の出力例

```
Using CPython 3.11.x
Creating virtual environment at: .venv
Resolved 47 packages in 0.5ms
Installed 47 packages in 8.2s
+ bm3d==4.0.2
+ ipywidgets==8.1.5
+ japanize-matplotlib==1.1.3
+ jupyterlab==4.3.6
+ matplotlib==3.9.4
+ numpy==1.26.4
+ PyWavelets==1.5.0
+ scikit-image==0.22.0
+ scipy==1.13.1
...
```

このような出力が出れば完了です。

7.2.2 2 回目以降の実行

すでに環境が揃っている場合は、`uv sync` を再実行しても差分だけが更新されます。不足しているパッケージがあれば追加されます。

```
$ uv sync
Resolved 47 packages in 0.5ms
Audited 47 packages in 1ms
```

7.3 セットアップの確認

正しくインストールされたか確認します。

インストール済みパッケージの確認

```
$ uv pip list
Package                                Version
-----                                -
bm3d                                    4.0.2
```

```
japanize-matplotlib      1.1.3
jupyterlab                4.3.6
matplotlib                3.9.4
numpy                     1.26.4
...
```

numpy、jupyterlab、bm3d などが表示されれば正しくインストールされています。

uv.lock の役割

uv.lock ファイルがあることで、先輩・後輩・教員の全員がまったく同じバージョンのライブラリを使えます。「自分の環境では動いたのに先輩の環境では動かなかった」という問題を防ぐための仕組みです。

第 8 ノートブックの実行方法

8.1 VS Code でノートブックを開く（推奨）

8.1.1 事前準備：Jupyter 拡張機能のインストール

VS Code を初めて使う場合のみ必要です。

1. VS Code を開く
2. 左側の拡張機能アイコン (`Ctrl` + `Shift` + `X`) をクリック
3. 検索欄に Jupyter と入力
4. Jupyter (Microsoft 製) をインストール

8.1.2 フォルダを開いてノートブックを実行する

1. VS Code で File → Open Folder を選ぶ
2. 2026 年度 B4 課題引継ぎ フォルダを開く
3. 左側のファイルツリーから `.ipynb` ファイルをクリック
4. 初回はカーネル選択が求められるので `.venv/bin/python` を選ぶ

カーネルの選び方

ノートブックを開いたとき、右上に「カーネルを選択」または Python のバージョンが表示されます。クリックして一覧から `.venv/bin/python`（または Python 3.11.x（`'.venv'`））を選んでください。

一覧に出ない場合は先に `uv sync` を実行してから VS Code を開き直してください。

8.1.3 セルの実行

操作	方法
セルを実行する	<code>Shift</code> + <code>Enter</code>
セルを実行（その場にとどまる）	<code>Ctrl</code> + <code>Enter</code>
全セルを一括実行	ノートブック上部の Run All ボタン
カーネルを再起動	Restart ボタンまたは <code>F1</code> で検索

8.2 課題ノートブックの一覧

課題	ファイル	内容
No.1	No.1_FID/makesignal.ipynb	FID 信号生成とノイズ付加
No.1	No.1_FID/heikatsu.ipynb	移動平均による平滑化
No.2	No.2_FFT1D/FFT1D.ipynb	1 次元 FFT
No.2	No.2_FFT1D/kukei.ipynb	矩形パルスの FFT
No.2	No.2_FFT1D/sft.ipynb	スライディング FFT
No.3	No.3_FFT2D/FFT2D.ipynb	2 次元 FFT (MRI k 空間)
No.4	No.4_DFT_exercise/DFT_template.ipynb	DFT 手動実装 (演習)
No.5	No.5_CompressedSensing/cs_tv.ipynb	圧縮センシング (TV 法)
No.5	No.5_CompressedSensing/cs_wavelet.ipynb	圧縮センシング (Wavelet)
No.6	No.6_Denoising/add_noise.ipynb	ノイズ付加
No.6	No.6_Denoising/denoise_bm3d.ipynb	BM3D デノイジング
No.7	No.7_Metrics/psnr_ssim.ipynb	PSNR・SSIM 計算
No.8	No.8_CT/ct_pipeline.ipynb	CT フィルタード逆投影

No.4 は演習形式

DFT_template.ipynb は空白セルに自分でコードを書く演習です。解答例は DFT_answer.ipynb にあります。まず自分で考えてから見るようにしてください。

No.6 の WNNM はオプション

`denoise_wnnm.ipynb` は発展課題です。256×256 画像で数分かかるため、まず `denoise_bm3d.ipynb` を完了させてから取り組んでください。

8.3 JupyterLab を使う場合（任意）

ブラウザ上でノートブックを操作したい場合は JupyterLab を使えます。

```
$ cd ~/NMR-lab-kadai年度/2026課題引継ぎ_B4
$ uv run jupyter lab
```

uv run を必ずつける

`jupyter lab` だけで起動すると仮想環境の外から起動してしまい、ライブラリが見つかりません。必ず `uv run jupyter lab` と打ってください。

起動するとターミナルに URL が表示されます。ブラウザで `http://localhost:8888` を開いてください。（WSL では自動でブラウザが開かない場合があります。その場合は URL 全体をコピーして貼り付けてください。）

終了するときはターミナルで `Ctrl` + `C` を押します。

ブラウザのタブを閉じるだけでは終了しない

タブを閉じてもサーバーは動き続けます。必ずターミナルで `Ctrl` + `C` を押して終了してください。

第9 よくあるエラーと対処

9.1 uv: command not found

原因：uv がインストールされていないか、PATH が通っていません。

対処 1：ターミナルを閉じて開き直す。

対処 2：PATH を今すぐ反映させる。

```
$ source ~/.bashrc
```

対処 3：それでも解決しない場合は PATH を手動で追加する。

```
$ echo 'export PATH="$HOME/.local/bin:$PATH"' \  
>> ~/.bashrc  
$ source ~/.bashrc
```

最後に確認：

```
$ uv --version  
uv 0.6.x (linux x86_64 ...)
```

9.2 No pyproject.toml found

原因：pyproject.toml のあるフォルダにいません。

対処：正しいフォルダに移動してから実行する。

```
$ cd ~/NMR-lab-kadai年度/2026課題引継ぎ_B4
$ uv sync
```

フォルダに `pyproject.toml` があるか確認：

```
$ ls pyproject.toml
pyproject.toml
```

`pyproject.toml` と表示されれば正しいフォルダにいます。

9.3 JupyterLab でカーネルが見つからない

原因：`uv run` をつけずに JupyterLab を起動しています。

対処：ターミナルで `Ctrl` + `C` を押して一度終了し、正しいコマンドで起動し直す。

```
$ cd ~/NMR-lab-kadai年度/2026課題引継ぎ_B4
$ uv run jupyter lab
```

9.4 ModuleNotFoundError が出る

原因 1：`uv sync` が完了していない。

対処：`uv sync` を再実行する。

```
$ cd ~/NMR-lab-kadai年度/2026課題引継ぎ_B4
$ uv sync
```

原因 2：`uv run` なしで JupyterLab を起動している。→ 前節の対処と同じ手順で起動し直す。

9.5 `uv sync` がネットワークエラーで止まる

原因：インターネット接続が不安定、または学内ネットワークのプロキシが干渉しています。

対処：数分待ってから再実行する。uv sync は途中まで進んだ場合、次回実行時に続きから再開します。

```
$ uv sync
```

繰り返し発生する場合は教員・先輩に相談してください。

9.6 VS Code で「カーネルを起動できませんでした」と表示される

症状：ノートブックを開くと「Python 環境 'nmrlab-b4-2026 (3.12.x)」を使用できなくなったため、カーネルを起動できませんでした」と表示される。

原因：VS Code が以前選択したカーネルをキャッシュしているため、.venv/ を作り直したあとに古い参照が残ります。pyvenv.cfg を手動で書き換えても uv sync で元に戻るの、この方法では解決しません。

対処：uv sync で .venv/ を作り直したあと、VS Code でカーネルを選び直す。

1. ターミナルで .venv/ を削除して再構築する

```
$ cd ~/NMR-lab-kadai年度/2026課題引継ぎ_B4
$ rm -rf .venv
$ uv sync
```

2. VS Code でノートブックを開き、右上の「カーネルを選択」をクリック
3. 「別のカーネルを選択」→「Python 環境」を選ぶ
4. 一覧から .venv/bin/python（または nmrlab-b4-2026 (3.12.x)）を選択する

一覧に出ない場合

「Python 環境の一覧を更新」をクリックするか、VS Code を開き直してから再度選択してください。

9.7 Jupyter で「Python 3.12.x が使用できなくなった」と表示される

原因：フォルダ内に古い `.venv/` が残っており、そこに保存されていた Python のパスが現在の環境に存在しません。（配布ファイルを古いバージョンから上書き展開した場合に起こります。）

対処：`.venv/` を削除してから `uv sync` を再実行する。

```
$ cd ~/NMR-lab-kadai年度/2026課題引継ぎ_B4
$ rm -rf .venv
$ uv sync
```

`uv sync` が完了したら VS Code でフォルダを開き直し、カーネルを `.venv/bin/python` に選び直してください。

9.8 `uv sync` 後に Permission denied が出る

原因：フォルダを Windows の共有ドライブから直接コピーした場合、ファイルの実行権限が失われることがあります。

対処 1：`.venv/` を削除して再構築する（最も確実）。

```
$ cd ~/NMR-lab-kadai年度/2026課題引継ぎ_B4
$ rm -rf .venv
$ uv sync
```

対処 2：フォルダ全体の権限を修正する。

```
$ chmod -R u+rw \
~/NMR-lab-kadai年度/2026課題引継ぎ_B4
```

git clone で取得した場合は発生しにくい

`git clone` で正しく取得した場合、パーミッション問題はほとんど起きません。問題が続く場合は `.venv/` を削除して `uv sync` を再実行してください（第 6 章参照）。

9.9 ブラウザが自動で開かない

原因：WSL から Windows のブラウザを自動起動できない場合があります。

対処：ターミナルに表示された URL を手動でコピーしてブラウザに貼り付ける。

```
[I ServerApp] Jupyter Server is running at:  
[I ServerApp] http://localhost:8888/lab  
?token=abc123def456...   <- これを全部コピーする
```


第 10 補足知識

10.1 .venv/ フォルダの扱い

.venv/ は各自の PC で `uv sync` を実行すれば再生成できます。以下のことを覚えておいてください。

- 共有フォルダに含める必要はない
- メールや zip でやり取りする必要はない
- 誤って削除しても `uv sync` で復元できる
- リポジトリの `.gitignore` で除外済み

10.2 環境を作り直す

環境が壊れたと感じたときは `.venv/` を削除して作り直せます。

```
$ cd ~/NMR-lab-kadai年度/2026課題引継ぎ_B4
$ rm -rf .venv
$ uv sync
```

10.3 新しいパッケージを追加する

演習の範囲を超えて新しいライブラリを試したい場合：

```
$ uv add パッケージ名
```

例えば `pandas` を追加する場合：

```
$ uv add pandas
Resolved 49 packages in 0.3ms
Installed 2 packages in 1.2s
+ pandas==2.2.3
+ python-dateutil==2.9.0
```

pyproject.toml と uv.lock が自動的に更新されます。

10.4 スクリプトを直接実行する

ノートブックを使わず Python スクリプト (.py ファイル) を直接実行したい場合も uv run を使います。

```
$ cd ~/NMR-lab-kadai年度/2026課題引継ぎ_B4/No.1_FID
$ uv run python makesignal.py
```

10.5 MATLAB とのデータ互換

このプロジェクトでは scipy.io を通じて MATLAB の .mat ファイルを読み書きできます。

Listing 10.1 .mat ファイルの読み込み例

```
import scipy.io

# MATLAB の .mat ファイルを読む
data = scipy.io.loadmat("Signal.mat")

# Python の配列を .mat ファイルに保存する
scipy.io.savemat("result.mat", {"signal": signal})
```

10.6 インストール済みパッケージの確認

```
$ uv pip list
```

第 11 セットアップ手順まとめ（チートシート）

11.1 最初の 1 回だけ行う作業

```
$ curl -LsSf https://astral.sh/uv/install.sh | sh
```

ターミナルを閉じて開き直す（または `source ~/.bashrc`）。

```
$ cd ~  
$ git clone \  
  https://github.com/shiru2/NMR-lab-kadai.git
```

```
$ cd \  
  ~/NMR-lab-kadai年度/2026課題引継ぎ_B4  
$ uv sync
```

11.2 毎回の作業

VS Code（推奨）：VS Code で 2026 年度_B4 課題引継ぎ フォルダを開くと、カーネルが自動で `.venv` に設定されます。

```
$ code \  
  ~/NMR-lab-kadai年度/2026課題引継ぎ_B4
```

JupyterLab（任意）：

```
$ cd \  
~/NMR-lab-kadai年度/2026課題引継ぎ_B4  
$ uv run jupyter lab
```

ブラウザで `http://localhost:8888` を開いて作業する。終了するときはターミナルで `Ctrl` + `C`。

11.3 よく使うコマンドまとめ

コマンド	内容
<code>uv --version</code>	uv のバージョン確認
<code>uv sync</code>	仮想環境を作成・更新する
<code>uv run python スクリプト名</code>	Python スクリプトを実行する
<code>uv run jupyter lab</code>	JupyterLab を起動する（任意）
<code>source .venv/bin/activate</code>	仮想環境を手動で有効化する
<code>uv pip list</code>	インストール済みパッケージを確認する
<code>uv add パッケージ名</code>	新しいパッケージを追加する
<code>git pull</code>	課題ファイルを最新版に更新する
<code>uv self update</code>	uv 自体をアップデートする

11.4 トラブル時の確認フロー

課題ノートブックが動かないときは以下の順番で確認してください。

1. `uv sync` を実行してパッケージが揃っているか確認する
2. VS Code で 2026 年度_B4 課題引継ぎ フォルダが開かれているか確認する（サブフォルダだけでは不十分）
3. カーネルエラーが出たら `rm -rf .venv && uv sync` を試す
4. それでも解決しない場合は第 9 章を参照する

参考：uv 公式ドキュメント <https://docs.astral.sh/uv/>

第 12 研究での活用：uv init / activate / VS Code

B4 課題では既存の `pyproject.toml` に対して `uv sync` するだけでしたが、自分の研究プログラムを書くときは**ゼロからプロジェクトを作る**必要があります。この章では研究用途の `uv` の使い方と、スクリプトを実行する 3 つの方法、VS Code との連携を説明します。

12.1 uv init：新しいプロジェクトを作る

自分で研究プログラムを書くときは `uv init` から始めます。

```
$ cd ~
$ uv init myresearch
$ cd myresearch
$ ls
hello.py  pyproject.toml  .python-version  README.md
```

`uv init` は以下のファイルを自動生成します。

`pyproject.toml` プロジェクト設定ファイル（空の依存関係リストで生成される）

`.python-version` 使用する Python バージョンを固定するファイル

`hello.py` サンプルスクリプト（削除して構わない）

12.1.1 パッケージを追加する

必要なパッケージを追加

```
$ uv add numpy matplotlib scipy
Resolved 10 packages in 0.3ms
Installed 10 packages in 2.1s
+ numpy==1.26.4
+ matplotlib==3.9.4
```

```
+ scipy==1.13.1
```

uv add を実行すると：

- .venv/ が自動作成される
- pyproject.toml の dependencies に追記される
- uv.lock が自動生成・更新される

12.1.2 パッケージを削除する

```
$ uv remove scipy
```

12.2 Python スクリプトを実行する 3 つの方法

仮想環境内の Python でスクリプトを実行する方法は 3 つあります。

方法	コマンド	特徴
方法 (1)	uv run python script.py	activate 不要・最も推奨
方法 (2)	source .venv/bin/activate して から python script.py	従来の venv と同じ操作
方法 (3)	.venv/bin/python script.py	パスを直接指定・activate 不要

12.2.1 方法 (1) uv run (推奨)

```
$ uv run python script.py
```

- activate の必要がない
- プロジェクトフォルダ (pyproject.toml がある場所) であれば自動的に .venv/ を使う
- uv sync が済んでいれば問題なく動く
- このマニュアルで **最も推奨する方法**

12.2.2 方法 (2) activate してから実行

```
$ source .venv/bin/activate
(.venv) $ python script.py
(.venv) $ python another.py # activate は一度だけでよい
(.venv) $ deactivate        # 終わったら解除
```

- `source .venv/bin/activate` で仮想環境を「有効化」する
- 有効化するとプロンプトが `(.venv) $` に変わる
- 以降は普通に `python` と打つだけで仮想環境内の Python が使われる
- `deactivate` で元の状態に戻る
- 従来の `venv` 操作に慣れている人はこちらでも問題ない

activate のスコープはターミナル 1 つ

`source .venv/bin/activate` の効果はそのターミナルウィンドウだけです。新しいターミナルを開いたら、再度 `activate` する必要があります。`uv run` はこの問題がないため、ターミナルをまたぐ作業では便利です。

12.2.3 方法 (3) .venv/bin/python を直接指定

```
$ .venv/bin/python script.py
```

- `.venv/` 内の Python 実行ファイルを直接パス指定して実行する
- `activate` も `uv run` も不要
- シェルスクリプトや Makefile から呼び出すときに使いやすい
- VS Code の Python インタープリタ設定でもこのパスを指定する

どの Python が使われているか確認する

```
$ which python # システム Python のパス
/usr/bin/python3

$ uv run which python # 仮想環境内 Python のパス
/home/username/myresearch/.venv/bin/python
```



```
$ .venv/bin/python --version # 直接確認
Python 3.11.x
```

12.3 VS Code との連携

VS Code でプロジェクトを開くと、右下に Python のバージョンが表示されます。ここで正しい仮想環境の Python インタープリタを選ぶことが重要です。

12.3.1 インタープリタの選択

1. VS Code でプロジェクトフォルダを開く (File → Open Folder)
2. `Ctrl` + `Shift` + `P` でコマンドパレットを開く
3. Python: Select Interpreter と入力して選択
4. 一覧に `.venv/bin/python` が表示されたら選択する

`.venv` が一覧に出ない場合

プロジェクトフォルダが VS Code で開かれていないか、`uv sync` がまだ完了していない可能性があります。先に `uv sync` を実行してから VS Code を開き直してください。

12.3.2 VS Code の統合ターミナルで実行する

VS Code 内のターミナル (`Ctrl` + ``) はインタープリタを設定済みであれば `activate` 状態で開きます。

```
(.venv) $ python script.py # 仮想環境が自動で有効
```

12.3.3 Python ファイルを「実行」ボタンで動かす

インタープリタを設定した後、`.py` ファイルを開いて右上の  ボタン (Run ボタン) または `F5` で実行できます。このとき仮想環境が自動的に使われます。

12.3.4 Jupyter Notebook を VS Code で開く

VS Code の Jupyter 拡張機能を使うと、ブラウザを使わずに直接 `.ipynb` ファイルを開けます。

1. 拡張機能マーケットプレイスで Jupyter をインストール
2. `.ipynb` ファイルを VS Code で開く
3. 右上のカーネル選択で `.venv/bin/python` を選ぶ

JupyterLab vs VS Code のどちらを使う？

ツール	向いている場面
JupyterLab (ブラウザ)	グラフをインタラクティブに確認したいとき、ノートブック主体の作業
VS Code	<code>.py</code> スクリプトとノートブックを行き来したいとき、コード補完・デバッグを活用したいとき
どちらを使っても仮想環境さえ正しく設定されていれば同じ結果が出ます。	

12.4 研究プロジェクトの典型的な流れ

自分の研究プログラムを始めるときの典型的な手順をまとめます。

研究プロジェクトの開始から実行まで

```
# 1. プロジェクトを作成
$ uv init myresearch
$ cd myresearch

# 2. 必要なパッケージを追加
$ uv add numpy matplotlib scipy

# 3a. uv run で実行 (推奨)
$ uv run python analysis.py

# 3b. activate して実行 (従来スタイル)
$ source .venv/bin/activate
(.venv) $ python analysis.py
(.venv) $ deactivate
```

```
# 3c. パスを直接指定して実行
$ .venv/bin/python analysis.py
```

どれを使うか迷ったら

- 毎回コマンドで実行する → **方法 (1)** `uv run`
- VS Code でコーディングしながら実行する → **VS Code + 方法 (2)** (自動 activate)
- シェルスクリプトや自動化に組み込む → **方法 (3)** パス直接指定

12.5 よく使う uv コマンド一覧

12.5.1 Python バージョン管理

```
$ uv python list          # 利用可能な Python の一覧
$ uv python list --only-installed # インストール済みのみ
$ uv python pin 3.11      # .python-version に 3.11 を固定
```

`uv python pin` を実行すると `.python-version` ファイルにバージョンが書き込まれ、以降 `uv sync` や `uv run` はそのバージョンを優先して使います。共同作業するときや、Python バージョンを明示したいときに使います。

12.5.2 依存関係の確認

```
$ uv tree
myresearch v0.1.0 |——
  matplotlib v3.9.4 | |——
    numpy v1.26.4 | |——
    ... |——
  scipy v1.13.1 |——
    numpy v1.26.4 (*)
```

`uv tree` でどのパッケージが何に依存しているかを確認できます。(*) は別の場所で既に表示済みであることを示します。

12.5.3 requirements.txt へのエクスポート

pip を使っている人に環境を共有したり、サーバーにデプロイするときは requirements.txt 形式に変換できます。

```
$ uv export -o requirements.txt
$ uv export --no-hashes -o requirements.txt
```

12.5.4 ロックファイルだけを更新する

パッケージをインストールせずに uv.lock だけを最新の状態に更新したいときは uv lock を使います。

```
$ uv lock          # uv.lock を更新（インストールはしない）
$ uv lock --upgrade # 全パッケージを最新版に更新
```

12.5.5 uvx：ツールを一時的に実行する

uvx はインストールせずにツールを一時実行できます (pipx run 相当)。コードフォーマッタやリンタを手軽に試すときに便利です。

```
$ uvx ruff check .      # コードチェック () ruff
$ uvx black script.py   # コードフォーマット () black
$ uvx mypy script.py    # 型チェック () mypy
```

常用するツールはプロジェクトに追加する方法もあります：

```
$ uv add --dev ruff      # 開発依存として追加
$ uv run ruff check .
```

12.5.6 キャッシュの管理

uv はダウンロードしたパッケージをキャッシュに保存するため、2 回目以降のインストールが高速です。ディスク容量が逼迫したときはキャッシュを削除できます。

```
$ uv cache dir      # キャッシュの場所を確認  
$ uv cache clean    # キャッシュを全削除
```

付録 A 参考リンク集

A.1 uv 公式ドキュメント

<https://docs.astral.sh/uv/>

英語のみですが、コマンドの使い方や設定の詳細はここが最も正確です。このマニュアルで扱った内容に対応するページを以下にまとめます。

A.2 本マニュアルとの対応

このマニュアルの章	対応する公式ドキュメント
第 4 章 uv のインストール	https://docs.astral.sh/uv/getting-started/installation/
第 3 章 uv の仕組み（入門）	https://docs.astral.sh/uv/getting-started/first-steps/
第 3 章 pyproject.toml / uv sync / uv.lock	https://docs.astral.sh/uv/concepts/projects/
第 12 章 uv init・プロジェクト管理	https://docs.astral.sh/uv/guides/projects/
第 12 章 uv run でスクリプト実行	https://docs.astral.sh/uv/guides/scripts/
第 12 章 Python バージョン管理	https://docs.astral.sh/uv/concepts/python-versions/
第 8 章 JupyterLab との連携	https://docs.astral.sh/uv/guides/integration/jupyter/

VS Code との連携について

uv の公式ドキュメントには VS Code 専用のページはありません。VS Code 側の設定 (Python インタープリタの選択など) は VS Code 公式ドキュメントを参照してください。

<https://code.visualstudio.com/docs/python/environments>